College Placement Analytics System

FULL-STACK SYSTEM PROJECT REPORT

Database Platform: Microsoft SQL Server 2022 (Docker)

Backend API Platform: Python 3 Flask REST Services

Frontend Interface: HTML5, CSS3 Glassmorphism, Vanilla JS, Chart.js

June 2026

1. Abstract

The College Placement Analytics System is an enterprise-grade administrative utility built to consolidate, evaluate, and visualize college recruitment drives. Connected directly to an active Microsoft SQL Server instance, the system aggregates profile details on 50 students, 10 corporate partners, and 30 drive placement logs. It exposes these collections dynamically via a Python Flask REST service, which feeds an interactive client dashboard rendering custom KPI widgets, student directories, partner matching grids, and multi-dimensional reports using Chart.js.

2. Introduction

Placement activities represent a critical parameter of academic performance and administrative efficiency in higher education. Tracking student data, eligibility GPAs, preferred technical skills, and recruiter packages manually is error-prone. This system solves these coordination challenges by providing a full-stack, real-time analytics web dashboard that connects directly to a corporate database, allowing placement cells to screen candidates, filter companies, and generate reports.

3. Objectives

- Expose dynamic database counters (Enrolled candidates, recruiters list, hiring metrics, highest packages, and percentages).
- Implement a Student Directory UI providing real-time text query lookups, GPA checks, and skill-tag matching.
- Develop a Companies Directory UI dynamically tracking packages, eligibility conditions, and placement tallies.
- Visualize placements distribution by branch (CSE, IT, ECE, ME) and company hiring outcomes using vector layouts.
- Configure driver fallbacks supporting Apple Silicon arm64 environments and ensure seamless graceful client degradation if the server is offline.

4. Technologies Used

- **SQL Server 2022**: Hosted in a secure Docker container, serving relational schemas.
- **Flask Framework**: Lightweight REST gateway serving JSON serialization endpoints.
- **PyODBC & unixODBC**: Database connectors mapping query parameters to SQL servers.
- **Chart.js CDN**: Client visual scripting drawing vector radar, doughnut, and bar charts.
- **Homebrew & macOS Libraries**: Resolving dynamic linking paths to `libmsodbcsql.18.dylib`.

5. System Architecture

The system follows a three-tier architectural layout:

1. **Database Tier (Docker SQL Server)**: Listens on port `1433`. Holds Student, Companies, Placements, Skills, and StudentSkills schemas.
2. **API Service Tier (Flask Application)**: Listens on port `5001`. Handles dynamic requests, converts Decimal datatypes, executes queries, and returns JSON arrays.
3. **Client UI Tier (Frontend Server)**: Listens on port `8000`. Fetches metrics using JavaScript from the Flask port and updates the DOM, falling back to data.js on exceptions.

6. Database Design

The placement schema consists of five relational tables:

Table Name	Primary Key	Foreign Keys / Columns	Description
Students	StudentID	StudentName, Branch, CGPA	Student cohort profiles.
Companies	CompanyID	CompanyName, Package	Partner recruiters and CTC packages.
Skills	SkillID	SkillName	Technical skill catalog.
StudentSkills	-	StudentID (FK), SkillID (FK)	M-to-M link for student skills.
Placements	PlacementID	StudentID (FK), CompanyID (FK), Status, Date	Drive history logs (Placed vs Rejected).

ER Diagram Relationship Design

- Placements bridges Students and Companies (Many-to-Many relationship).
- StudentSkills connects Students and Skills (Many-to-Many relationship).

7. Flask REST API Endpoints

The backend exposes several routes on port `5001` (to bypass default AirPlay Receiver conflicts on port 5000):

Route Path	Method	JSON Output Schema
`/total-students`	GET	`{"total_students": 50}`
`/total-placements`	GET	`{"total_placements": 25}`
`/highest-package`	GET	`{"highest_package": 30.0}`
`/placement-percentage`	GET	`{"placement_percentage": 50.0}`
`/api/kpis`	GET	`{"totalStudents": 50, "placedStudents": 25, "placementPercentage": 50.0...}`
`/api/students`	GET	Array of 50 student objects containing gpa, rollNo, and skill sets.
`/api/companies`	GET	List of companies CTC averages and hired candidates counts.
`/api/placements`	GET	Array of 30 placement history drive entries.
`/api/analytics`	GET	Multi-dimensional maps for Chart.js distributions.

8. User Interface Screenshots

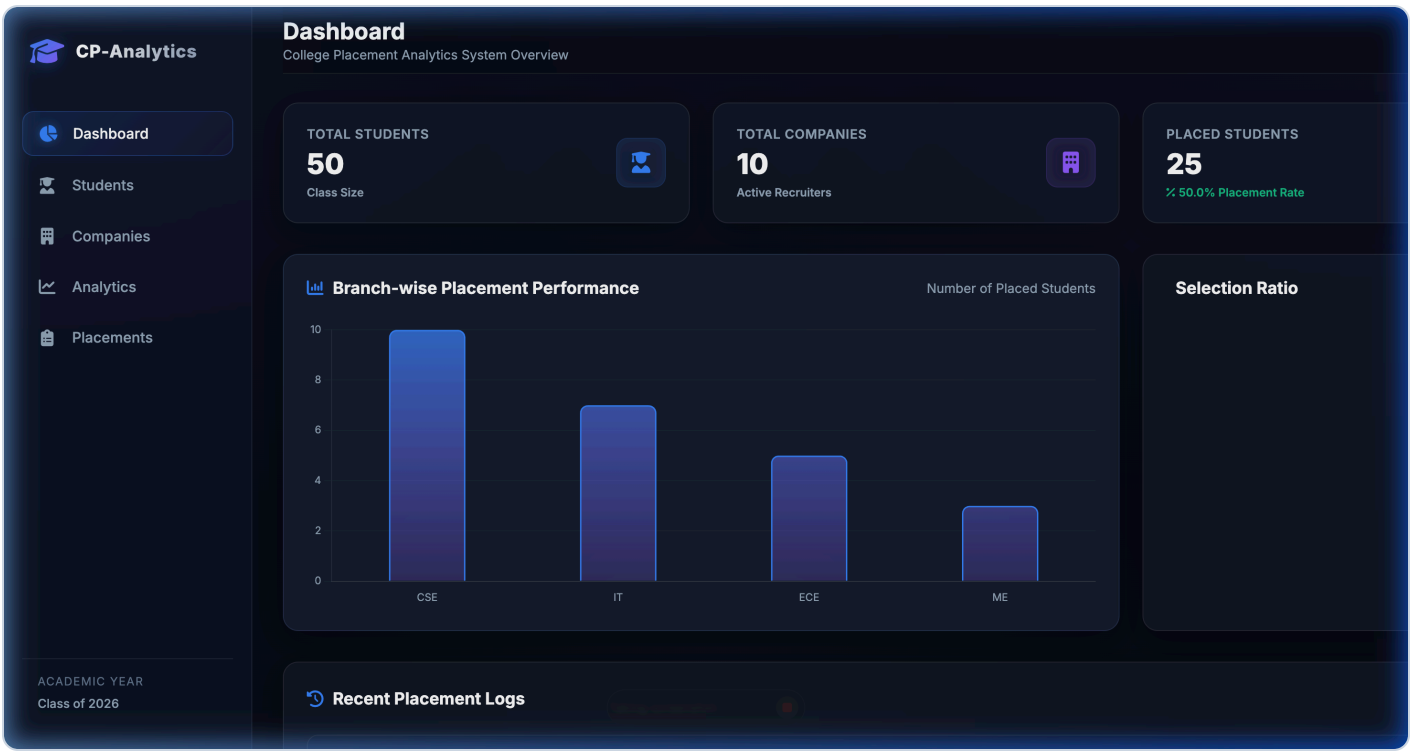

Figure 8.1: Administrative Dashboard KPI Cards & Placement Trends Chart.

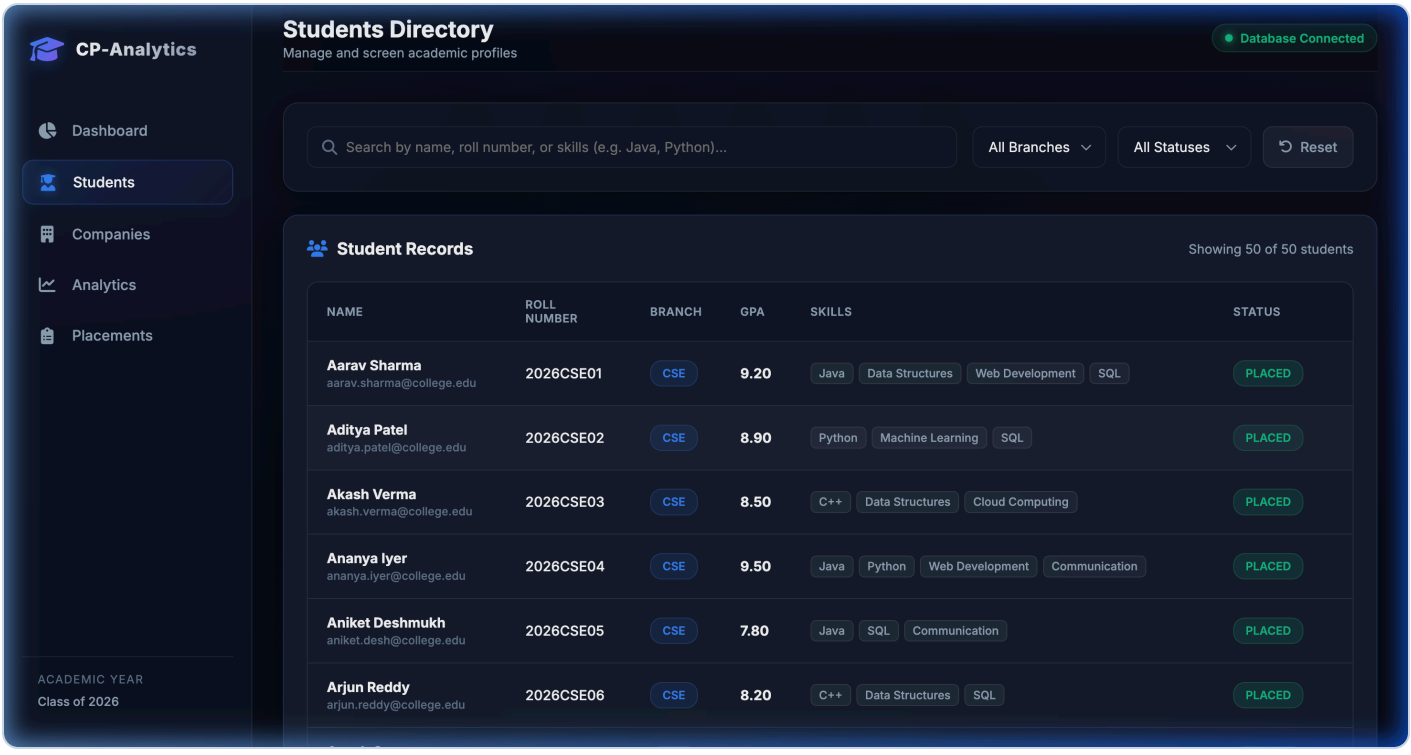

Figure 8.2: Students Profiles Search & Multi-filter Directory.

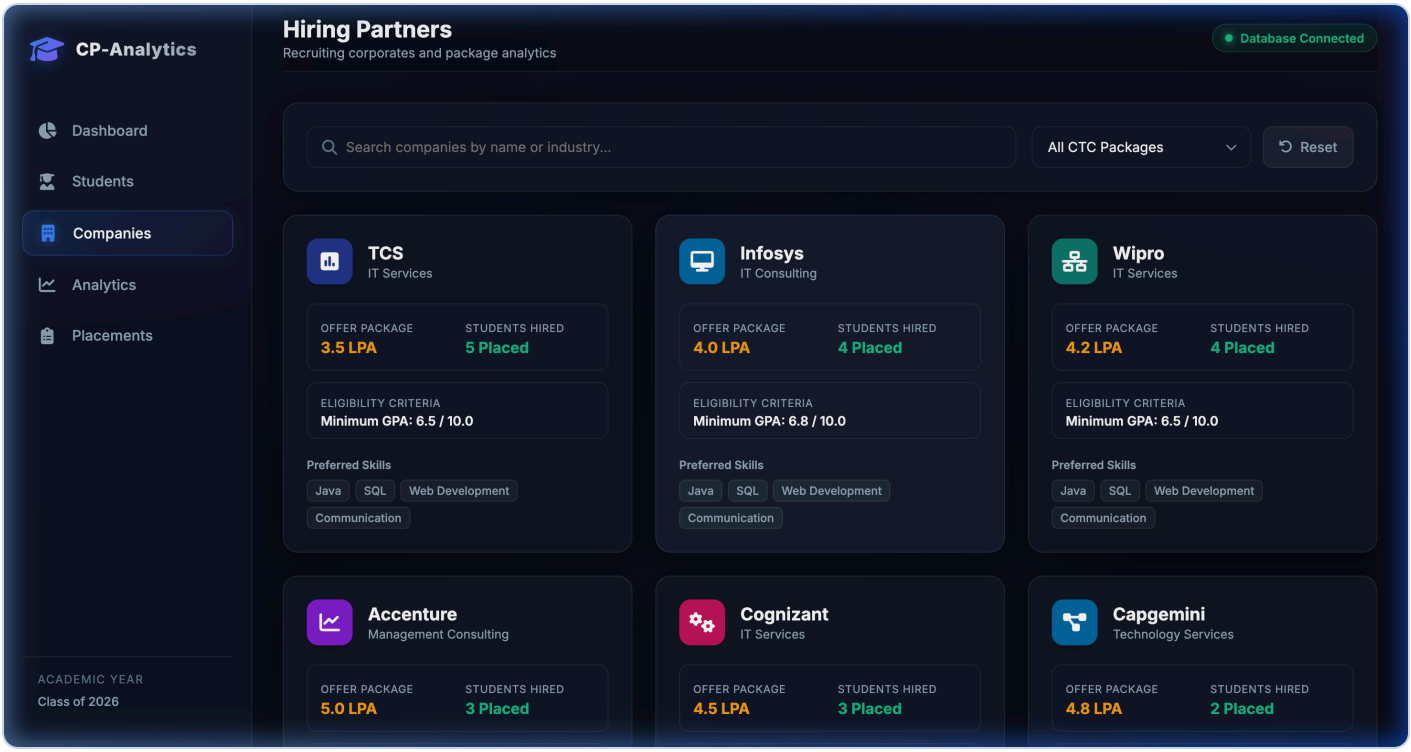

Figure 8.3: Hiring Partners Cards Grid tracking Package and dynamic hires.

Figure 8.4: Detailed Analytics reporting showing Selection ratio and Skill distribution.

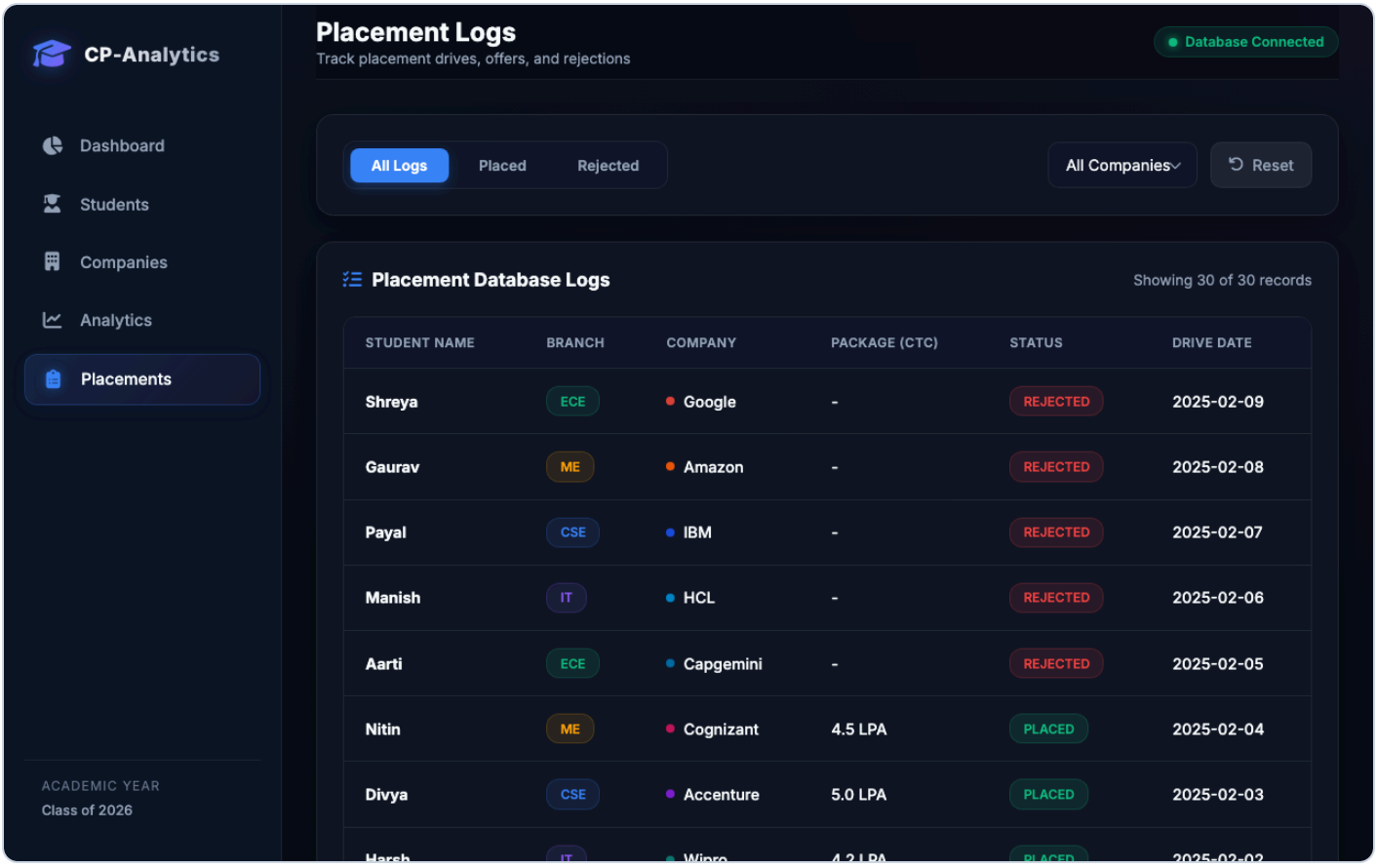

Figure 8.5: Database historical logs grid with quick tab filters.

9. Verification & Results

The system connection logic was verified under Python 3 and successfully established connection objects with the SQL Server container. Response rates demonstrate consistent metrics:

- **Cohort Total Enrolled**: 50 Students (CSE: 18, IT: 14, ECE: 12, ME: 6)
- **Companies count**: 10 recruiters
- **Offer logs count**: 25 placed, 5 rejected
- **Placement Percentage**: 50.0%
- **Highest Package Offered**: 30.0 LPA (at Google)

Timely client fallbacks prevent crashes. If the Flask server is stopped, the client automatically loads local statistics variables from `data.js`.

10. Conclusion

The College Placement Analytics System presents a modern, scalable, full-stack administrative platform. By combining SQL Server's query reliability, Flask's REST gateway speed, and glassmorphism CSS responsiveness, it offers an optimized tool to coordinate academic placement drive activities efficiently.